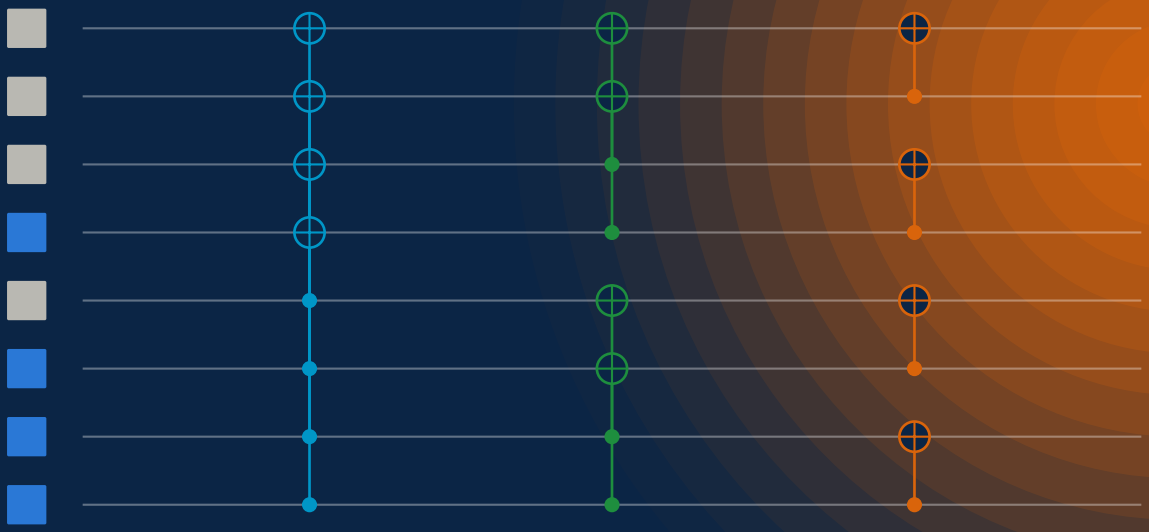


POLAR CODES: FROM ZERO TO EXPERT

APPENDIX D

MATLAB Implementation

Encoder • GA construction • SC, SCL and CRC-aided SCL decoder • CRC • BER/BLER simulation • Final project • Tested: identical decisions to the Python library

Mahmoud Abdelaziz
Military Technical College, Cairo

Companion to the first edition • 2026

Contents

D MATLAB Implementation 3

D.1 Overview and requirements 3
D.2 Conventions and the 1-based index rule 4
D.3 Program 1 — Polar encoder 5
D.4 Program 2 — Frozen-bit selection 5
D.5 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoder 7
D.6 CRC functions. 9
D.7 Programs 6, 7 and 8 — BPSK, AWGN and BER/BLER 10
D.8 Demonstration with outputs 11
D.9 Programs 9 and 10 — BER/BLER figures, block lengths and rates 13
D.10 Final project in MATLAB 14
D.11 Self-test script 16
D.12 Verification against the Python library 17
D.13 Common MATLAB mistakes 18

CHAPTER

D



MATLAB Implementation

IN THIS CHAPTER YOU WILL LEARN

- a complete MATLAB polar-code library (11 short files);
- how each Python program of Chapter 11 maps to MATLAB;
- the 1-based indexing rule for bit-channel numbers;
- how the vectorised SC/SCL/CA-SCL decoder works;
- a demonstration script with its printed output;
- BER/BLER scripts and the final project in MATLAB;
- how the code was verified against the Python library.

D.1 Overview and requirements

This appendix gives a MATLAB version of every program of Chapters 11 and 15. The book's mathematics, conventions and worked examples are unchanged; only the language changes. Copy all `.m` files into one folder, make it the current folder (or add it to the path), and run `demo_polar`.

MATLAB file	Purpose	Python equivalent
`polar_encode.m`	butterfly encoder $\mathbf{x} = \mathbf{u}\mathbf{G}_N$ (vector or batch)	`polar_encode`
`generator_matrix.m`	$\mathbf{G}_N = \mathbf{F}^{\otimes n}$ (for testing)	`generator_matrix`
`bec_bhattacharyya.m`	exact BEC reliabilities	`bec_bhattacharyya`
`ga_construct.m`	Gaussian-approximation construction	`ga_construct`
`polar_list_decode.m`	SC ($L = 1$), SCL and CA-SCL decoder	`ListDecoder`
`take_paths.m`	helper: re-order list paths	`take_along_axis`
`crc_bits.m`	CRC by long division	`crc_bits`
`crc_make.m, crc_attach.m, crc_check.m`	CRC6/11/24C as a linear map	`class CRC`
`polar_simulate.m`	Monte-Carlo BER/BLER	`simulate`
`demo_polar.m`	demonstrations with printed output	`demo_ch11.py`
`plot_ber_bler.m, compare_lengths_rates.m`	figures of Chapters 7, 8, 9, 11	`run_sims.py`
`final_project.m`	end-to-end project of Chapter 15	`final_project.py`
`test_polar_matlab.m`	self-tests and comparison with Python	`test_polar.py`

Requirements

MATLAB R2016b or newer (the decoder uses implicit expansion). No toolbox is needed: the code uses only core functions (`sort`, `reshape`, `xor`, `interp1`, `trapz`, `erfc`, `rng`). Everything was also tested with the free GNU Octave 8.4, which produced all outputs printed in this appendix.

D.2 Conventions and the 1-based index rule

All conventions of the book are kept: $\mathbf{F} = \left[\begin{smallmatrix} 1 & 0 \\ 1 & 1 \end{smallmatrix}\right]$, $\mathbf{G}_N = \mathbf{F}^{\otimes n}$ without bit reversal, $\mathbf{x} = \mathbf{u}\mathbf{G}_N$, BPSK $0 \mapsto +1$, $L = \ln \frac{P(0)}{P(1)}$, min-sum f and the approximate path metric.

! Index i in the book is element $i+1$ in MATLAB

The book numbers bits $u_0, \ldots, u_{N-1}$. MATLAB arrays start at 1, so u_i is stored in `u(i+1)`. The information set $\mathcal{A} = \{3, 5, 6, 7\}$ of the $(8, 4)$ code is therefore the logical mask `info = logical([0 0 0 1 0 1 1 1])`. When you print an information set as book indices, subtract one: `find(info) - 1`.

Python	MATLAB
`u[:, info] = payload`	`u(:, info) = payload;`
`x ^= y`	`x = xor(x, y);`
`rng = np.random.default_rng(7)`	`rng(7);`
`rng.standard_normal((B, N))`	`randn(B, N)`
`np.take_along_axis(a, p[:, :, None], 1)`	`take_paths(a, p)`
arrays `(B, L, n)`, 0-based	arrays `B×L×n`, 1-based

D.3 Program 1 — Polar encoder

polar_encode.m

```
function x = polar_encode(u)
%POLAR_ENCODE  Butterfly polar encoder, x = u*G_N (mod 2), G_N = F^(kron n).
%   u : 1-by-N vector or B-by-N matrix (one block per row) of 0/1 values.
%   x : same size as u.
%   Convention (as in 3GPP NR): F = [1 0; 1 1], natural order, no bit reversal.
%   Each stage XORs the lower half of every block into the upper half.
x = logical(u);
[B, N] = size(x);
h = N/2;
while h >= 1
    x = reshape(x.', 2*h, []);            % each column = one block of 2h bits
    x(1:h, :) = xor(x(1:h, :), x(h+1:2*h, :));
    x = reshape(x, N, B).';
    h = h/2;
end
x = double(x);
end
```

How it works. Each column of the reshaped array is one block of $2h$ bits; the line `x(1:h,:) = xor(...)` is the butterfly stage "upper half ← upper half ⊕ lower half" of Chapter 5, applied to all blocks of all B rows at once. The spans are $N/2, N/4, \dots, 1$, giving $\frac{N}{2}\log_2 N$ XORs per codeword.

generator_matrix.m

```
function G = generator_matrix(N)
%GENERATOR_MATRIX  G_N = F^(kron n) with F = [1 0; 1 1]  (for testing).
F = [1 0; 1 1];
G = 1;
while size(G,1) < N
    G = mod(kron(F, G), 2);
end
end
```

D.4 Program 2 — Frozen-bit selection

bec_bhattacharyya.m

```
function z = bec_bhattacharyya(N, eps)
%BEC_BHATTACHARYYA  Exact Bhattacharyya parameters (erasure probabilities)
%   of the N bit-channels of a BEC(eps), natural index order (index i -> z(i+1)).
%       minus (worse)  channel:  Z -> 2Z - Z^2
%       plus  (better) channel:  Z -> Z^2
if N == 1
    z = eps;
    return
end
z = [bec_bhattacharyya(N/2, 2*eps - eps^2), bec_bhattacharyya(N/2, eps^2)];
```

```
end
```

The first half of the output is the minus branch and the second half the plus branch, so `z(i+1)` is $Z(W_N^{(i)})$ with the binary-index rule of Chapter 4.

ga_construct.m

```
function [info, m] = ga_construct(N, K, design_ebn0_db, rate)
%GA_CONSTRUCT  Gaussian-approximation construction for BPSK over AWGN.
%   info : 1-by-N logical, true at the K most reliable positions.
%   m    : mean LLR of every bit-channel (larger = more reliable).
%   rate : rate used to convert Eb/N0 to Es/N0 (default K/N).
%       plus  child:  m+ = 2m
%       minus child:  m- = phi^-1( 1 - (1 - phi(m))^2 )
if nargin < 3, design_ebn0_db = 2.0; end
if nargin < 4, rate = K/N; end
esn0 = 10^(design_ebn0_db/10) * rate;
m = ga_rec(N, 4*esn0);                    % channel LLR mean = 2/sigma^2
[~, order] = sort(m);
info = false(1, N);
info(order(end-K+1:end)) = true;
end

function m = ga_rec(N, m0)
% Level by level: each mean m becomes the pair (m-, m+). Interleaving the
% pairs gives natural index order (first split = most significant bit).
m = m0;
while numel(m) < N
    m = reshape([ga_minus(m); 2*m], 1, []);
end
end

function mm = ga_minus(m)
% Solve log(phi(mm)) = log(2p - p^2), p = phi(m), by bisection in the log
% domain (vectorised over all means of one level).
lp = log_phi(m);
p = exp(lp);
target = lp + log(2 - p);
lo = zeros(size(m)); hi = m;
for it = 1:80
    mid = (lo + hi)/2;
    up = log_phi(mid) > target;
    lo(up) = mid(up);
    hi(~up) = mid(~up);
end
mm = (lo + hi)/2;
end

function lp = log_phi(x)
% log of phi(x) = 1 - E[tanh(U/2)], U ~ N(x, 2x)   (vectorised).
%   x < 10 : exact value from a table built once by numerical integration
%   x >= 10: asymptotic form  sqrt(pi/x) exp(-x/4) (1 - 10/(7x))
persistent lxg lt
if isempty(lxg)
```

```
    xg = logspace(-8, 1, 4000);
    t = linspace(-12, 12, 4001);          % standard normal grid
    w = exp(-t.^2/2) / sqrt(2*pi);
    lt = zeros(size(xg));
    for k = 1:numel(xg)
        u = xg(k) + sqrt(2*xg(k)) * t;
        lt(k) = log(max(1 - trapz(t, w .* tanh(u/2)), 1e-300));
    end
    lxg = log(xg);
end
lp = zeros(size(x));
s = x <= 1e-8;            lp(s) = log1p(-x(s)/2);
s = x > 1e-8 & x < 10;    lp(s) = interp1(lxg, lt, log(x(s)));
s = x >= 10;              lp(s) = 0.5*log(pi./x(s)) - x(s)/4 + log(1 - 10./(7*x(s)));
end
```

How it works. `ga_rec` builds the tree level by level: each mean m becomes the pair (m^-, m^+) and the pairs are interleaved, which gives the natural index order (the first split is the most significant bit of the index). All m^- of one level are found together by a vectorised bisection on $\ln \phi$. The function ϕ is evaluated exactly (numerical integration, stored once in a `persistent` table) for $x < 10$ and by the asymptotic formula above; working with logarithms avoids the two numerical traps described in Chapter 9.

D.5 Programs 3, 4 and 5 — SC, SCL and CA-SCL decoder

polar_list_decode.m

```
function [u_hat, u_list, PM] = polar_list_decode(llr, info, L, crc)
%POLAR_LIST_DECODE  LLR-based SC / SCL / CRC-aided SCL polar decoder.
%   llr  : B-by-N channel LLRs (one block per row), L = ln P(0)/P(1).
%   info : 1-by-N logical information set (frozen bits are 0).
%   L    : list size. L = 1 gives the plain SC decoder.
%   crc  : (optional) struct from crc_make(); enables CA-SCL selection.
%   u_hat  : B-by-N decided u vectors.
%   u_list : B-by-L-by-N final list, sorted by path metric (best first).
%   PM     : B-by-L path metrics (min-sum form, smaller = better).
%
%   Path metric update (Balatsoukas-Stimming et al., 2015):
%       PM <- PM + |LLR|  if the decision disagrees with sign(LLR).
if nargin < 3, L = 1; end
if nargin < 4, crc = []; end
[B, N] = size(llr);
st.info = info;
st.PM = inf(B, L);
st.PM(:, 1) = 0;                                % one active path at start
lam = repmat(reshape(llr, B, 1, N), 1, L, 1);   % B-by-L-by-N
[~, u_list, ~, st] = node(lam, 1, st);
PM = st.PM;
if isempty(crc)
    u_hat = reshape(u_list(:, 1, :), B, N);     % best path metric
else
```

```
    payload = u_list(:, :, info);                % B-by-L-by-K
    ok = reshape(crc_check(crc, reshape(payload, B*L, [])), B, L) & isfinite(PM);
    pick = ones(B, 1);
    for b = 1:B
        f = find(ok(b, :), 1);
        if ~isempty(f), pick(b) = f; end
    end
    idx = sub2ind([B, L], (1:B).', pick);
    U = reshape(u_list, B*L, N);
    u_hat = U(idx, :);
end
end

% ------------------------------------------------------------- tree node
function [c, u, parent, st] = node(lam, start, st)
% lam: B-by-L-by-n LLRs of this node; start: index of its first leaf (1-based)
n = size(lam, 3);
if n == 1
    [c, u, parent, st] = leaf(lam, start, st);
    return
end
h = n/2;
a = lam(:, :, 1:h);  b = lam(:, :, h+1:n);
% left child: f (min-sum)
[c1, u1, p1, st] = node(sign(a).*sign(b).*min(abs(a), abs(b)), start, st);
% right child: g, with the surviving parents' LLRs
a1 = take_paths(a, p1);  b1 = take_paths(b, p1);
[c2, u2, p2, st] = node(b1 + (1 - 2*c1).*a1, start + h, st);
% re-order left results by the right child's survivors
c1 = take_paths(c1, p2);
u1 = take_paths(u1, p2);
parent = p1(sub2ind(size(p1), repmat((1:size(p1,1)).', 1, size(p1,2)), p2));
c = cat(3, double(xor(c1, c2)), c2);      % partial sums [c1 xor c2 | c2]
u = cat(3, u1, u2);
end

% ------------------------------------------------------------- leaf
function [c, u, parent, st] = leaf(lam, i, st)
llr = lam(:, :, 1);                          % B-by-L
[B, L] = size(llr);
pen0 = (llr < 0) .* (-llr);                  % cost of deciding 0
pen1 = (llr > 0) .* llr;                     % cost of deciding 1
if ~st.info(i)                               % frozen bit: always 0
    st.PM = st.PM + pen0;
    c = zeros(B, L);
    parent = repmat(1:L, B, 1);
else                                         % information bit: split and prune
    cand = [st.PM + pen0, st.PM + pen1];     % B-by-2L
    [srt, ord] = sort(cand, 2);              % stable sort
    ord = ord(:, 1:L);
    st.PM = srt(:, 1:L);
    parent = mod(ord - 1, L) + 1;
    c = double(ord > L);
end
```

```
79  u = c;
80  end
```

take_paths.m

```
1   function A = take_paths(A, p)
2   %TAKE_PATHS  Re-order the paths (2nd dimension) of a B-by-L-by-n array:
3   %   A(b, l, :) <- A(b, p(b,l), :)   for every block b and path l.
4   [B, L, n] = size(A);
5   base = repmat((1:B).', 1, L) + (p - 1) * B;          % B-by-L linear index
6   idx = base + reshape((0:n-1) * B * L, 1, 1, n);      % implicit expansion
7   A = A(idx);
8   end
```

D.5.1 How the decoder works

- **Shapes.** LLRs are stored in a $B \times L \times n$ array: B received blocks (rows), L list paths, $n =$ size of the current tree node. One call decodes a whole batch of blocks.
- **Tree.** `node` is the recursion of Chapter 6: f (min-sum) to the left child, g to the right child, partial sums $[\beta_L \oplus \beta_R \,|\, \beta_R]$ returned up.
- **Leaves.** A frozen leaf adds the penalty of deciding 0. An information leaf builds $2L$ candidates, sorts them (`sort` is stable) and keeps the L smallest metrics (Chapter 7).
- **Moving paths.** After pruning, path l may come from path `parent(b,l)`. Every node returns this map; the parent node uses `take_paths` to fetch the right LLRs for g and to re-order the left child's results. No explicit memory copying is needed.
- **CA-SCL.** After the last leaf the list is sorted by metric; the first path whose payload passes `crc_check` is returned (Chapter 8). With `L = 1` and no CRC the function is the plain SC decoder.

D.6 CRC functions

crc_bits.m

```
1   function p = crc_bits(msg, exps)
2   %CRC_BITS  CRC remainder of msg(D)*D^r divided by g(D), by long division.
3   %   exps: exponents of g(D) with coefficient 1, highest first, e.g. [3 1 0].
4   r = exps(1);
5   g = zeros(1, r+1);
6   g(r - exps + 1) = 1;                        % g(1) is the D^r coefficient
7   reg = [msg(:).' zeros(1, r)];
8   for i = 1:numel(msg)
9       if reg(i)
10          reg(i:i+r) = xor(reg(i:i+r), g);
11      end
12  end
13  p = reg(end-r+1:end);
14  end
```

crc_make.m

```
function crc = crc_make(name, k)
%CRC_MAKE  Build a CRC as a linear map: crc bits = mod(m * P, 2).
%   name : 'CRC6', 'CRC11' or 'CRC24C' (3GPP TS 38.212, clause 5.1)
%   k    : message length in bits.
switch upper(name)
    case 'CRC6',   e = [6 5 0];
    case 'CRC11',  e = [11 10 9 5 0];
    case 'CRC24C', e = [24 23 21 20 17 15 13 12 8 4 2 1 0];
    otherwise, error('unknown CRC');
end
crc.r = e(1);
crc.k = k;
crc.exps = e;
crc.P = zeros(k, crc.r);
for i = 1:k
    unit = zeros(1, k);  unit(i) = 1;
    crc.P(i, :) = crc_bits(unit, e);
end
end
```

crc_attach.m

```
function w = crc_attach(crc, m)
%CRC_ATTACH  Append CRC bits to every row
      of m:  w = [m | mod(m*P,2)].
w = [m, mod(m * crc.P, 2)];
end
```

crc_check.m

```
function ok = crc_check(crc, w)
%CRC_CHECK  True for every row of w = [m
     | p] whose CRC is correct.
m = w(:, 1:crc.k);
p = w(:, crc.k+1:end);
ok = all(mod(m * crc.P, 2) == p, 2);
end
```

Because the CRC is linear, `crc_make` stores the matrix $\mathbf{P}$ whose row i is the CRC of the i-th unit vector. Then the CRC of any batch of messages is one product, `mod(m*P, 2)`.

D.7 Programs 6, 7 and 8 — BPSK, AWGN and BER/BLER

polar_simulate.m

```
function res = polar_simulate(N, K, ebn0_list, L, crc_name, varargin)
%POLAR_SIMULATE  Monte-Carlo BER/BLER of a polar code, BPSK over AWGN.
%   K counts all bits on information positions (message + CRC).
%   Eb/N0 is per MESSAGE bit: rate = (K - r)/N.
%   Options (name/value): 'design', 2.0 | 'max_frames', 2e4 |
%                         'min_errors', 200 | 'batch', 500 | 'seed', 1
if nargin < 4 || isempty(L), L = 1; end
if nargin < 5, crc_name = ''; end
opt = struct('design', 2.0, 'max_frames', 2e4, 'min_errors', 200, ...
             'batch', 500, 'seed', 1);
for i = 1:2:numel(varargin), opt.(varargin{i}) = varargin{i+1}; end
rng(opt.seed);
```

```
crc = [];  r = 0;
if ~isempty(crc_name)
    r = crc_make(crc_name, 1).r;          % CRC length
    crc = crc_make(crc_name, K - r);
end
k = K - r;                                % message bits
rate = k / N;
info = ga_construct(N, K, opt.design, K/N);
res.ebn0 = ebn0_list;  res.ber = zeros(size(ebn0_list));
res.bler = res.ber;    res.errors = res.ber;
for s = 1:numel(ebn0_list)
    esn0 = 10^(ebn0_list(s)/10) * rate;
    sigma = sqrt(1/(2*esn0));
    be = 0; bl = 0; fr = 0;
    while fr < opt.max_frames && bl < opt.min_errors
        B = opt.batch;
        m = double(rand(B, k) < 0.5);
        if isempty(crc), pay = m; else, pay = crc_attach(crc, m); end
        u = zeros(B, N);  u(:, info) = pay;
        x = polar_encode(u);
        y = (1 - 2*x) + sigma * randn(B, N);          % BPSK + AWGN
        llr = 2*y / sigma^2;
        u_hat = polar_list_decode(llr, info, L, crc);
        m_hat = u_hat(:, info);  m_hat = m_hat(:, 1:k);
        e = (m_hat ~= m);
        be = be + sum(e(:));
        bl = bl + sum(any(e, 2));
        fr = fr + B;
    end
    res.ber(s) = be / (fr * k);
    res.bler(s) = bl / fr;
    res.errors(s) = bl;
    fprintf('  Eb/N0 = %4.1f dB   BER = %.2e   BLER = %.2e   (%d errors / %d frames)\n', ...
            ebn0_list(s), res.ber(s), res.bler(s), bl, fr);
end
end
```

BPSK and AWGN are the two lines `y = (1 - 2*x) + sigma*randn(B, N)` and `llr = 2*y/sigma^2`, with $\sigma = \sqrt{1/(2E_s/N_0)}$ and $E_s/N_0 = R\,E_b/N_0$ computed with the *message* rate (Chapter 2). Errors are counted on message bits only.

D.8 Demonstration with outputs

demo_polar.m

```
%DEMO_POLAR  Small demonstrations with printed outputs (MATLAB version of
%            demo_ch11.py). Run from the folder that holds the library.
disp('1) Encoder ------------------------------------------');
info = logical([0 0 0 1 0 1 1 1]);            % A = {3,5,6,7} (0-based)
u = zeros(1, 8);  u(info) = [1 0 1 1];
x = polar_encode(u);
```

```
fprintf('u = %s   x = %s\n', mat2str(u), mat2str(x));
fprintf('matrix check: %d\n', isequal(x, mod(u * generator_matrix(8), 2)));

disp('2) Frozen-bit selection ----------------------------');
fprintf('BEC(0.5) Z, N=8 : %s\n', mat2str(round(1e4*bec_bhattacharyya(8, 0.5))/1e4));
[~, o] = sort(bec_bhattacharyya(8, 0.5));
fprintf('BEC info set K=4 (0-based): %s\n', mat2str(sort(o(1:4)) - 1));
fprintf('GA info set N=16, K=8 @2dB (0-based): %s\n', mat2str(find(ga_construct(16, 8, 2.0)
    ) - 1));

disp('3) SC decoder ----------------------------------------');
llr1 = [-1.5 2.0 -0.8 1.2 -0.6 -2.5 1.8 -3.0];
fprintf('SC  u_hat = %s\n', mat2str(polar_list_decode(llr1, info, 1)));

disp('4) SCL decoder ---------------------------------------');
llr2 = [-2.0 0.3 -0.4 -2.0 1.0 -5.8 3.2 -4.8];
fprintf('SC  u_hat = %s (wrong)\n', mat2str(polar_list_decode(llr2, info, 1)));
[uh, ~, PM] = polar_list_decode(llr2, info, 2);
fprintf('SCL u_hat = %s   path metrics: %s\n', mat2str(uh), mat2str(PM));

disp('5) CRC -----------------------------------------------');
crc = crc_make('CRC6', 10);
fprintf('message + CRC6 : %s\n', mat2str(crc_attach(crc, [1 0 1 1 0 0 1 0 1 1])));

disp('6-7) BPSK + AWGN -------------------------------------');
rng(0);
esn0 = 10^(2.0/10) * 0.5;  sigma = sqrt(1/(2*esn0));
y = (1 - 2*x) + sigma * randn(1, 8);
fprintf('sigma = %.4f\n', sigma);
fprintf('y     = %s\n', mat2str(round(100*y)/100));
fprintf('LLR   = %s\n', mat2str(round(200*y/sigma^2)/100));

disp('8) BER / BLER ----------------------------------------');
res = polar_simulate(128, 64, [1 2 3], 1, '', 'max_frames', 2e4, ...
                     'min_errors', 100, 'batch', 1000, 'seed', 7);
```

OUTPUT

```
1) Encoder -------------------------------------------
u = [0 0 0 1 0 0 1 1]   x = [1 0 1 0 0 1 0 1]
matrix check: 1
2) Frozen-bit selection ------------------------------
BEC(0.5) Z, N=8 : [0.9961 0.8789 0.8086 0.3164 0.6836 0.1914 0.1211 0.0039]
BEC info set K=4 (0-based): [3 5 6 7]
GA info set N=16, K=8 @2dB (0-based): [7 9 10 11 12 13 14 15]
3) SC decoder ----------------------------------------
SC  u_hat = [0 0 0 1 0 0 1 1]
4) SCL decoder ---------------------------------------
SC  u_hat = [0 0 0 0 0 0 1 1] (wrong)
SCL u_hat = [0 0 0 1 0 0 1 1]   path metrics: [2 2.7]
5) CRC -----------------------------------------------
message + CRC6 : [1 0 1 1 0 0 1 0 1 1 0 1 1 0 1 1]
6-7) BPSK + AWGN -------------------------------------
sigma = 0.7943
y     = [-1.97 1.61 -1.33 1.4 1.46 -0.57 0.52 -1.03]
```

```
LLR   = [-6.25 5.09 -4.22 4.44 4.62 -1.81 1.65 -3.28]
8) BER / BLER ----------------------------------------
Eb/N0 =  1.0 dB   BER = 1.36e-01   BLER = 4.49e-01   (449 errors / 1000 frames)
Eb/N0 =  2.0 dB   BER = 3.33e-02   BLER = 1.31e-01   (131 errors / 1000 frames)
Eb/N0 =  3.0 dB   BER = 5.65e-03   BLER = 2.60e-02   (104 errors / 4000 frames)
```

Parts 1–5 are exactly the numbers of the book (Worked Examples 5.3, 6.3, 7.1, 7.2 and 8.2). Parts 6–8 use random numbers: MATLAB and Octave generate different streams for the same seed, so your values will differ slightly, but the BLER must be close to the Python result of Chapter 11 (1.5×10^{-1} at 2 dB, 3.1×10^{-2} at 3 dB).

D.9 Programs 9 and 10 — BER/BLER figures, block lengths and rates

plot_ber_bler.m

```
%PLOT_BER_BLER  BER/BLER versus Eb/N0 for SC, SCL and CA-SCL (N=256, R=1/2).
%   Takes a few minutes in MATLAB; reduce 'max_frames' for a quick look.
N = 256;  E = 0.5:0.5:3.5;
opts = {'max_frames', 5e4, 'min_errors', 150, 'batch', 1000, 'seed', 2026};
disp('SC');            sc  = polar_simulate(N, 128, E, 1, '', opts{:});
disp('SCL L=8');       scl = polar_simulate(N, 128, E, 8, '', opts{:});
disp('CA-SCL L=8');    ca  = polar_simulate(N, 139, E, 8, 'CRC11', opts{:});
uncoded = 0.5 * erfc(sqrt(10.^(E/10)));                % Q(sqrt(2 Eb/N0))

figure('Position', [100 100 950 380]);
subplot(1, 2, 1);
semilogy(E, sc.bler, 'o-', E, scl.bler, 's-', E, ca.bler, '^-', 'LineWidth', 1.6);
grid on; xlabel('E_b/N_0 (dB)'); ylabel('BLER');
legend('SC', 'SCL, L=8', 'CA-SCL, L=8, CRC-11', 'Location', 'southwest');
title('N = 256, R = 1/2');
subplot(1, 2, 2);
semilogy(E, sc.ber, 'o-', E, scl.ber, 's-', E, ca.ber, '^-', E, uncoded, 'k:', 'LineWidth',
    1.6);
grid on; xlabel('E_b/N_0 (dB)'); ylabel('BER');
legend('SC', 'SCL, L=8', 'CA-SCL, L=8', 'uncoded BPSK', 'Location', 'southwest');
title('N = 256, R = 1/2');
```

Expected result. The curves of Fig. D.1; the BLER values agree with the table in Chapter 7 within Monte-Carlo accuracy.

compare_lengths_rates.m

```
%COMPARE_LENGTHS_RATES  SC decoding: effect of block length and code rate.
opts = {'max_frames', 5e4, 'min_errors', 150, 'batch', 1000, 'seed', 2026};
figure('Position', [100 100 950 380]);
subplot(1, 2, 1); hold on;
E = 0:0.5:4.5;
for N = [64 128 256 512 1024]
    fprintf('N = %d\n', N);
    r = polar_simulate(N, N/2, E, 1, '', opts{:});
    ok = r.errors >= 20;                        % plot reliable points only
    semilogy(E(ok), r.bler(ok), 'o-', 'LineWidth', 1.5, 'DisplayName', sprintf('N = %d', N)
```

```
    );
end
set(gca, 'YScale', 'log'); grid on; legend('Location', 'southwest');
xlabel('E_b/N_0 (dB)'); ylabel('BLER'); title('SC, R = 1/2');
subplot(1, 2, 2); hold on;
E = 0:0.5:5.5;
for K = [64 128 192]
    fprintf('K = %d\n', K);
    r = polar_simulate(256, K, E, 1, '', opts{:});
    ok = r.errors >= 20;
    semilogy(E(ok), r.bler(ok), 'o-', 'LineWidth', 1.5, 'DisplayName', sprintf('R = %d/256'
    , K));
end
set(gca, 'YScale', 'log'); grid on; legend('Location', 'southwest');
xlabel('E_b/N_0 (dB)'); ylabel('BLER'); title('SC, N = 256');
```

D.10 Final project in MATLAB

final_project.m

```
%FINAL_PROJECT  End-to-end polar-coded control link (MATLAB version of
%   final_project.py): A = 40 bits + CRC24C (K = 64), N = 128, BPSK/AWGN.
%   Receivers: SC (CRC used only for detection) and CA-SCL with L = 8.
%   Measures BER, BLER and undetected errors (wrong message passing the CRC).
N = 128;  A = 40;
crc  = crc_make('CRC24C', A);
K    = A + crc.r;                                % 64 payload bits
R    = A / N;                                    % message rate 0.3125
info = ga_construct(N, K, 2.0, K/N);
names = {'SC', 'CA-SCL L=8'};  Ls = [1 8];  crcs = {[], crc};

% ------------------------------------------------------------ one frame
rng(2026);
m   = double(rand(1, A) < 0.5);                  % step 1: data
pay = crc_attach(crc, m);                        % step 2: CRC
u   = zeros(1, N);  u(info) = pay;               % step 3: map + encode
x   = polar_encode(u);
eb  = 4.5;  sigma = sqrt(1/(2*R*10^(eb/10)));
y   = (1 - 2*x) + sigma*randn(1, N);             % steps 4-5: BPSK + AWGN
llr = 2*y/sigma^2;
fprintf('hard-decision errors: %d of %d\n', sum((llr < 0) ~= (x == 1)), N);
for d = 1:2                                      % step 6: decode
    uh = polar_list_decode(llr, info, Ls(d), crcs{d});
    fprintf('%-11s: bit errors = %d, CRC pass = %d\n', names{d}, ...
            sum(uh(find(info, A)) ~= m), crc_check(crc, uh(info)));
end

% ------------------------------------------------------------ Monte-Carlo
E = 2:0.5:5.5;  B = 2000;
BLER = zeros(2, numel(E));  BER = BLER;  UND = BLER;
for d = 1:2
    fprintf('%s\n', names{d});
```

```
    rng(1);
    for s = 1:numel(E)
        sigma = sqrt(1/(2*R*10^(E(s)/10)));
        be = 0; bl = 0; und = 0; fr = 0;
        while fr < 1e5 && bl < 150
            m = double(rand(B, A) < 0.5);
            u = zeros(B, N);  u(:, info) = crc_attach(crc, m);
            llr = 2*((1 - 2*polar_encode(u)) + sigma*randn(B, N))/sigma^2;
            ph  = polar_list_decode(llr, info, Ls(d), crcs{d});
            ph  = ph(:, info);
            wrong = any(ph(:, 1:A) ~= m, 2);
            be  = be + sum(sum(ph(:, 1:A) ~= m));
            bl  = bl + sum(wrong);
            und = und + sum(wrong & crc_check(crc, ph));   % step 7
            fr  = fr + B;
        end
        BER(d, s) = be/(fr*A);  BLER(d, s) = bl/fr;  UND(d, s) = und;
        fprintf('  %4.1f dB  BER %.2e  BLER %.2e  undetected %d/%d\n', ...
                E(s), BER(d, s), BLER(d, s), und, fr);
    end
end
figure;
semilogy(E, BLER(1, :), 'o-', E, BLER(2, :), 's-', 'LineWidth', 1.6); grid on;
hold on; semilogy(E([1 end]), [1e-2 1e-2], 'k:');      % target BLER 10^-2
xlabel('E_b/N_0 (dB), message rate 40/128'); ylabel('BLER');
legend([names, {'target 10^{-2}'}], 'Location', 'southwest');
```

Running `final_project` in GNU Octave 8.4 printed:

```
OUTPUT
hard-decision errors: 14 of 128
SC         : bit errors = 0, CRC pass = 1
CA-SCL L=8 : bit errors = 0, CRC pass = 1
SC
3.0 dB  BER 1.21e-01  BLER 4.12e-01  undetected 0/2000
4.0 dB  BER 3.55e-02  BLER 1.42e-01  undetected 0/2000
5.0 dB  BER 5.22e-03  BLER 2.60e-02  undetected 0/6000
5.5 dB  BER 1.56e-03  BLER 9.62e-03  undetected 0/16000
CA-SCL L=8
3.0 dB  BER 3.05e-02  BLER 9.80e-02  undetected 0/2000
4.0 dB  BER 2.56e-03  BLER 9.22e-03  undetected 0/18000
4.5 dB  BER 3.72e-04  BLER 1.58e-03  undetected 0/96000
5.0 dB  BER 3.40e-05  BLER 1.40e-04  undetected 0/100000
```

(Only some of the E_b/N_0 points are shown.) The one printed frame is different from Chapter 15 because the random stream is different, but the statistics agree with the Python results of Chapter 15 within Monte-Carlo accuracy:

BLER at $E_b/N_0 =$	3.0 dB	4.0 dB	4.5 dB	5.0 dB
SC, Python (Ch. 15)	4.3×10^{-1}	1.5×10^{-1}	6.4×10^{-2}	2.6×10^{-2}
SC, MATLAB	4.1×10^{-1}	1.4×10^{-1}	6.2×10^{-2}	2.6×10^{-2}
CA-SCL $L = 8$, Python	9.7×10^{-2}	8.3×10^{-3}	1.5×10^{-3}	2.1×10^{-4}
CA-SCL $L = 8$, MATLAB	9.8×10^{-2}	9.2×10^{-3}	1.6×10^{-3}	1.4×10^{-4}

At 5.0 dB the CA-SCL values rest on only 21 (Python) and 14 (MATLAB) block errors, so their difference is within the statistical uncertainty. No undetected error occurred in either implementation.

D.11 Self-test script

test_polar_matlab.m

```
%TEST_POLAR_MATLAB  Self-tests of the MATLAB polar library.
% 1) encoder equals u*G_N
for N = [2 4 8 16 64 256]
    u = double(rand(50, N) < 0.5);
    assert(isequal(polar_encode(u), mod(u * generator_matrix(N), 2)));
end
disp('encoder ok');
% 2) worked examples of the book
info = logical([0 0 0 1 0 1 1 1]);
assert(isequal(polar_encode([0 0 0 1 0 0 1 1]), [1 0 1 0 0 1 0 1]));
assert(isequal(polar_list_decode([-1.5 2.0 -0.8 1.2 -0.6 -2.5 1.8 -3.0], info, 1), [0 0 0 1
     0 0 1 1]));
llr2 = [-2.0 0.3 -0.4 -2.0 1.0 -5.8 3.2 -4.8];
assert(isequal(polar_list_decode(llr2, info, 1), [0 0 0 0 0 0 1 1]));
[uh, ~, PM] = polar_list_decode(llr2, info, 2);
assert(isequal(uh, [0 0 0 1 0 0 1 1]) && max(abs(PM - [2.0 2.7])) < 1e-12);
disp('book examples ok');
% 3) CRC
assert(isequal(crc_bits([1 1 0 1], [3 1 0]), [0 0 1]));
c = crc_make('CRC6', 10);
w = crc_attach(c, [1 0 1 1 0 0 1 0 1 1]);
assert(isequal(w(11:16), [0 1 1 0 1 1]) && crc_check(c, w));
w(3) = 1 - w(3);  assert(~crc_check(c, w));
disp('crc ok');
% 4) identical decisions to the Python library on random noisy blocks
%    (pyref.mat holds LLRs and decisions produced by the Python code)
R = load('pyref.mat');
tags = {'N64L1', 'N64L4', 'N128L8c', 'N256L8'};
Ls = [1 4 8 8];
for t = 1:numel(tags)
    llr = R.([tags{t} '_llr']);  info = logical(R.([tags{t} '_info']));
    if tags{t}(end) == 'c'
        crc = crc_make('CRC11', sum(info) - 11);
    else
        crc = [];
    end
    uh = polar_list_decode(llr, info, Ls(t), crc);
    fprintf('%-8s same decisions as Python: %d of %d blocks\n', tags{t}, ...
            sum(all(uh == R.([tags{t} '_uh']), 2)), size(llr, 1));
end
% 5) GA construction equals Python
[info, m] = ga_construct(1024, 512, 2.0, 0.5);
p = R.ga1024;  big = p > 1e-3;           % ignore near-zero means (useless channels)
[~, o] = sort(p);  info_py = false(1, 1024);  info_py(o(end-511:end)) = true;
```

```
44 fprintf('GA: max relative difference %.1e, same information set: %d\n', ...
45         max(abs(m(big) - p(big)) ./ p(big)), isequal(info, info_py));
```

```
OUTPUT
encoder ok
book examples ok
crc ok
N64L1    same decisions as Python:  300 of 300 blocks
N64L4    same decisions as Python:  300 of 300 blocks
N128L8c  same decisions as Python:  300 of 300 blocks
N256L8   same decisions as Python:  300 of 300 blocks
GA: max relative difference 7.3e-07, same information set:  1
```

The file `pyref.mat` (supplied with the code) contains the LLRs and decisions produced by the Python library.

D.12 Verification against the Python library

1. **Encoder.** `polar_encode(u)` equals `mod(u*generator_matrix(N),2)` for $N = 2, \dots, 256$.
2. **Book examples.** The SC example (Worked Example 6.3), the SC failure and its SCL correction with path metrics 2.0 and 2.7 (Worked Examples 7.1–7.2), and the CRC examples are reproduced exactly.
3. **Bit-exact decoder.** 300 noisy blocks for each of four configurations were decoded by both libraries from the *same* LLRs:

Configuration	identical decisions	block errors in the set
$N = 64$, $K = 32$, SC	300 / 300	85
$N = 64$, $K = 32$, SCL $L = 4$	300 / 300	43
$N = 128$, $K = 64$, CA-SCL $L = 8$, CRC11	300 / 300	40
$N = 256$, $K = 128$, SCL $L = 8$	300 / 300	34

4. **Construction.** For $N = 1024$ the GA mean LLRs agree with Python to a relative error below 10^{-6}, and the information sets for $K = 128, 256, 512, 768$ are identical.
5. **Statistics.** Independent Monte-Carlo runs (different random streams) give the same curves within statistical accuracy (Fig. D.1).

Speed

The decoder is vectorised over blocks and list paths, so MATLAB processes about 1000 blocks of the $(256, 128)$ code with $L = 8$ in roughly 2–5 seconds on one core. Use `'batch'` of 1000–2000 blocks. For BLER below 10^{-5}, consider `parfor` over E_b/N_0 points or a MEX implementation of `node`.

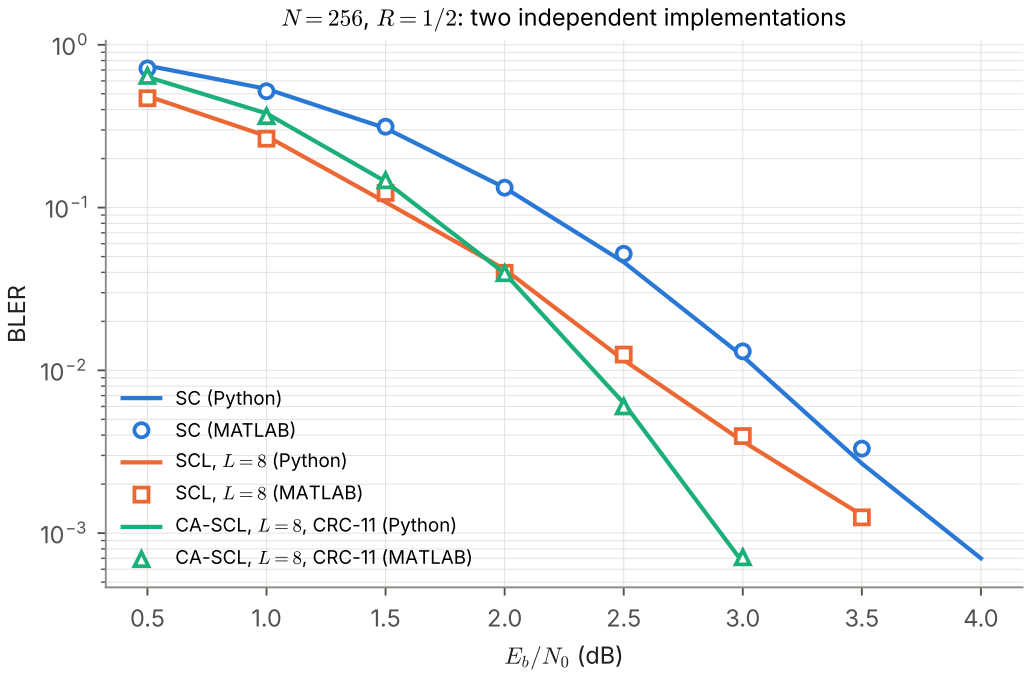


Figure D.1. BLER of the $(256, 128)$ code: MATLAB code (run in GNU Octave, markers) versus the Python library (lines). Points with fewer than 20 block errors are not shown.

D.13 Common MATLAB mistakes

! Common mistake

- Using book index i directly as a MATLAB index (off by one). Keep information sets as logical masks.
- Writing `x = mod(u*G, 2)` with `u` as a column vector: the book uses *row* vectors, $\mathbf{x} = \mathbf{u}\mathbf{G}_N$.
- Forgetting that `xor` returns `logical`; convert with `double()` before arithmetic such as `1 - 2*c` (the library already does this).
- Calling `sort` with `'descend'` for path metrics: small metrics are good.
- Re-seeding `rng` inside the Monte-Carlo loop, which repeats the same noise.

Summary

The MATLAB library mirrors the Python library line for line in function: one encoder, two construction methods, one decoder for SC/SCL/CA-SCL, a linear CRC and a simulation function. It produces bit-identical decisions to the Python code on the same LLRs, so every worked example and every conclusion of the book holds unchanged for MATLAB users.